Python Engineering

WEEK 02 - Data Structures | CLASS 05 - Lists & Tuples | EX13

Official structure change: we finish week_01_logic -> class_04_functions and begin **week_02_data_structures -> class_05_lists_tuples**.

EX13 - Store a trajectory with lists and tuples

So far the simulator computed each state and printed it. Once an iteration ended, the previous state was lost. In this exercise we learn to **store the complete history** of the simulation.

1. Why we need a data structure

```
t = 0.0, h = 0.0, v = 50.0
t = 0.1, h = 5.0, v = 49.019
t = 0.2, h = 9.902, v = 48.038
...
```

A normal variable stores one value at a time. A **list** stores many ordered values and can grow while the simulation runs.

2. First lists

```
times = []
heights = []
velocities = []

times.append(t)
heights.append(h)
velocities.append(v)
```

The `append()` method adds a new element to the end of a list. This turns the time loop into a trajectory-history generator.

3. Indexing

```
heights[0]    # first value
heights[1]    # second value
heights[-1]   # last value
len(heights)  # number of stored values
```

Python indexes from 0. Negative indices count backward from the end.

4. Tuples: a state we do not intend to modify

A tuple uses parentheses and is often used to group related values. Unlike a list, it is immutable.

```
state = (t, h, v)
t_state, h_state, v_state = state
```

A simulation can also store a list of complete states: `trajectory.append((t, h, v))`.

5. Two possible architectures

Option A	Option B
times = [] heights = [] velocities = []	trajectory = [] trajectory.append((t, h, v))

Very clear for plotting each variable	Compact: each element is one complete state

6. Your mission

Starting from the vertical-motion simulator, create `ex13_trajectory_lists_tuples.py`. Store the initial state and every state generated by Euler. No plots are required yet.

```
Required outputs:
- stored times
- stored heights
- stored velocities
- number of stored states
- first state
- last state
- maximum stored altitude
```

7. Test case

Start with h0=0, v0=50, g=9.81, dt=0.1. Verify that the first stored state matches the initial conditions and that the final stored state is the first point below the ground.

8. Learning goal

The key transition is from **computing one state** to **preserving a complete time series**. This is essential for plots, files, statistics, and NumPy later.

Learning tracker

Concept	EX13
Create lists	New
append()	New
Positive/negative indexing	New
len()	New
Tuples	New
Tuple unpacking	New
Simulation history	New